SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1711**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 Duration: 1 Hour Total Marks:50

Annexure A

CO1-AT1-Analytical Problem Solving

Code of Conduct:

I **S. Bhuvanesh (192521004)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

S. Bhuvanesh (192521004)

Annexure A

Analytical Problem Solving

5. 8-Puzzle with Heuristic Search

Solve the 8-puzzle from the start state below to the goal state, where _ is the blank.

Start				Goal		
2	8	3		1	2	3
1	6	4		8	_	4
7	_	5		7	6	5

i. Formulate the problem components.
ii. Define h_1 (number of misplaced tiles) and h_2 (sum of Manhattan distances), and compute both for the start state.
iii. Solve the puzzle with A* using h_2 and draw the search tree.
iv. Explain why h_2 dominates h_1.

i. Formulate the problem components

- Initial state:

$$\begin{matrix} 2 & 8 & 3 \\ 1 & 6 & 4 \\ 7 & _ & 5 \end{matrix}$$

- Goal state:

$$\begin{matrix} 1 & 2 & 3 \\ 8 & _ & 4 \\ 7 & 6 & 5 \end{matrix}$$

- State space: All valid arrangements of the eight tiles and one blank.
- Actions: Move the blank up, down, left, or right when the move is valid.
- Transition model: Swap the blank with an adjacent tile.
- Goal test: Check whether the current state matches the goal state.
- Path cost: Each move costs 1.
- Search algorithm: A* search, using $f(n) = g(n) + h(n)$.

ii. Define and calculate the heuristics

Heuristic 1: Misplaced tiles (h_1)

Counts the numbered tiles that are not in their goal positions. The blank is excluded.
Start state compared with the goal:

- Tile 2: misplaced
- Tile 8: misplaced
- Tile 1: misplaced
- Tile 6: misplaced

Therefore, $h_1 = 4$.

Heuristic 2: Manhattan distance (h_2)

The Manhattan distance is the sum of the horizontal and vertical distances of each numbered tile from its goal position.

Tile	Start position	Goal position	Distance
1	(2,1)	(1,1)	1
2	(1,1)	(1,2)	1
3	(1,3)	(1,3)	0
4	(2,3)	(2,3)	0
5	(3,3)	(3,3)	0
6	(2,2)	(3,2)	1
7	(3,1)	(3,1)	0
8	(1,2)	(2,1)	2

$$h_2 = 1 + 1 + 0 + 0 + 0 + 1 + 0 + 2 = \boxed{5}$$

Thus, the start state's heuristic values are $h_1 = 4$and $h_2 = 5$.

iii. Solve using A* with h_2

A* evaluates each state using:

$$f(n) = g(n) + h_2(n)$$

where $g(n)$is the cost from the start and $h_2(n)$is the Manhattan distance to the goal.
Optimal solution: 5 moves
Start state — $g = 0, h = 5, f = 5$
2
8
3
1
6
4
7

5
Move blank Up
2
8
3

1

_
4
7
6
5
Move blank Up
2

_
3
1
8
4
7
6
5
Move blank Left

_
2
3
1
8
4
7
6
5
Move blank Down
1
2
3

_
8
4
7
6
5
Move blank Right
1
2
3
8

_
4
7
6
5
Goal reached — total cost = 5
The values along the optimal path are:

State	$g(n)$	$h_2(n)$	$f(n)$

Start	0	5	5
After Up	1	4	5
After Up	2	3	5
After Left	3	2	5
After Down	4	1	5
After Right (Goal)	5	0	5

Search tree (optimal branch):

Start [g=0, h=5, f=5]
|
Up [1,4,5]
|
Up [2,3,5]
|
Left [3,2,5]
|
Down [4,1,5]
|
Right [5,0,5] --> GOAL

This is the optimal branch of the A* search tree. Other generated states are considered according to their f-values.

iv. Why does h_2 dominate h_1?

- h_1 counts each misplaced tile as 1.
- h_2 measures the actual horizontal and vertical distances of tiles from their goal positions.
- For every state, $h_2(n) \geq h_1(n)$.
- Both heuristics are admissible because neither overestimates the minimum remaining moves.
- Therefore, h_2 generally provides more information and can reduce the number of states expanded by A*.

6. Game Playing: Minimax with Alpha-Beta Pruning

In a two-player game, the root is a MAX node with three MIN children: B, C and D. Their leaf values, from left to right, are:

- B: 3, 12, 8
- C: 2, 4, 6
- D: 14, 5, 2

i. Apply the Minimax algorithm to find the value of the root and the best move.

ii. Apply alpha-beta pruning, mark the pruned branches, and show the α and β values at each node.

iii. Explain how move ordering affects how much alpha-beta can prune.

i. Apply Minimax

The root is a MAX node, and B, C, and D are MIN nodes.

Node	Leaf values	MIN value
B	3, 12, 8	3
C	2, 4, 6	2
D	14, 5, 2	2

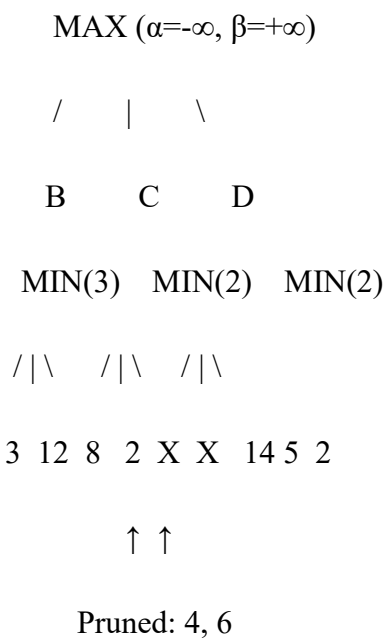
The root selects the maximum of these values:

$$\text{MAX}(3, 2, 2) = \boxed{3}$$

Best move: B. Root value: 3.

ii. Alpha-beta pruning

Alpha (α) is the best value MAX can guarantee. Beta (β) is the best value MIN can guarantee. Initially, $\alpha = -\infty$and $\beta = +\infty$.



Pruned: 4, 6

Step-by-step evaluation

1. Node B: Evaluate 3, 12, 8. Its minimum is 3. The root updates $\alpha = 3$.

2. Node C: The first leaf is 2, so $\beta = 2$. Since $\beta \leq \alpha (2 \leq 3)$, prune the remaining leaves 4 and 6.

3. Node D: Evaluate 14, then 5, then 2. Its minimum is 2. No additional leaves are saved because the cutoff condition is reached only at the final leaf.

Node	Alpha (α)	Beta (β)	Result
Root MAX	3	$+\infty$	3
B MIN	$-\infty$	3	3
C MIN	3	2	2; prune 4, 6
D MIN	3	2	2

The final root value remains 3, and B is the best move.

iii. Effect of move ordering

- Good move ordering examines promising moves first.
- This makes alpha or beta reach a cutoff value earlier.
- More branches can then be pruned, reducing computation.
- Poor move ordering may explore unnecessary branches before finding a cutoff.
- Alpha-beta pruning gives the same Minimax result regardless of move order, but the number of explored nodes changes.

9. Smart Traffic Signal Controller (Agent design)

A city wants an intelligent traffic signal at a four-way junction. It should adjust green-light durations using vehicle counts from cameras, give priority to emergency vehicles, and reduce average waiting time over the day.
i. Identify the most suitable type of agent (simple reflex, model-based reflex, goal-based, utility-based or learning agent), and justify your choice.
ii. Draw the agent architecture diagram.
iii. Write pseudocode for the agent's decision cycle.

i. Most suitable type of agent

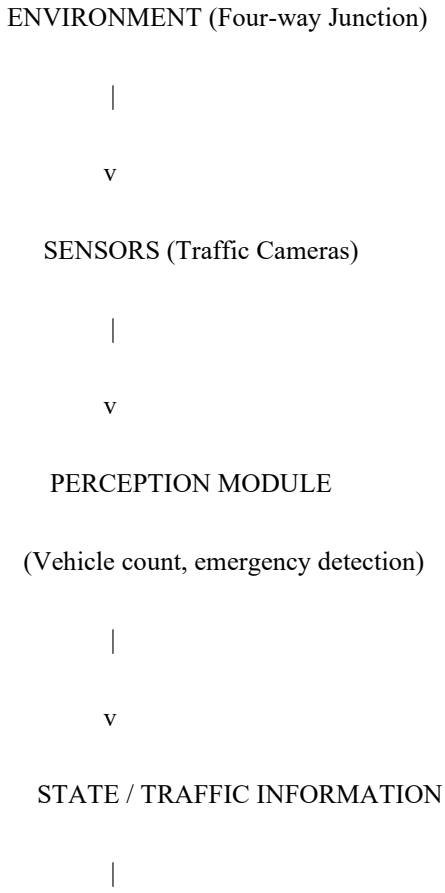
Utility-based agent is the most suitable choice.

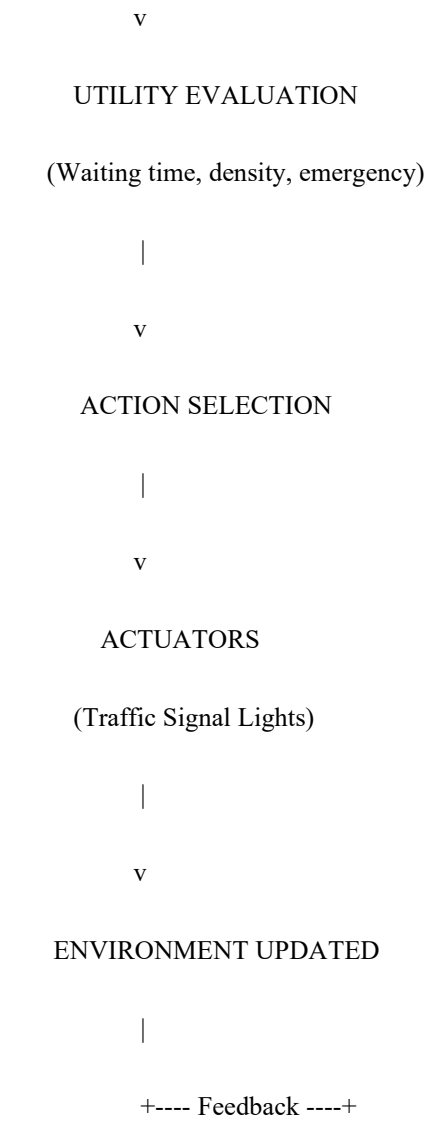
Justification:

- It uses camera data to count vehicles on each road.
- It adjusts green-light duration according to traffic density.
- It gives priority to emergency vehicles such as ambulances and fire engines.
- It aims to minimize average waiting time and traffic congestion.
- It compares possible actions and selects the one that provides the highest overall utility.

A learning component could also improve its decisions over time, but a utility-based agent is the best primary choice for the stated objectives.

ii. Agent architecture diagram





iii. Pseudocode for the decision cycle

```
START
WHILE traffic_signal_is_active:
    vehicle_count = read_traffic_cameras()
    emergency = detect_emergency_vehicles()
     IF emergency is detected:
       select_safe_emergency_priority()
```

```
    ELSE:
        FOR each road:
            calculate_traffic_utility()
        select_road_with_highest_utility()
    green_time = calculate_green_duration()
    green_time = apply_minimum_and_maximum_limits()
    safely_switch_signal()
    wait(green_time)

END WHILE
```

Result: The agent dynamically controls signal timing, prioritizes emergencies safely, and works to reduce waiting time.

10. Problem Formulation: Tower of Hanoi

There are three pegs (A, B, C), with three discs of different sizes stacked on peg A, largest at the bottom. All the discs must be moved to peg C. Only one disc can be moved at a time, and a larger disc may never be placed on a smaller one.

i. Formulate the problem: states, initial state, actions, transition model, goal test and path cost. ii. Draw the state space for the first three levels.

iii. Find the minimum sequence of moves and derive the general formula for n discs. iv. Would DFS or BFS be more suitable here? Justify your answer.

i. Formulate the problem

Component	Description
States	All legal arrangements of 3 discs on pegs A, B, and C
Initial state	All discs are stacked on peg A
Actions	Move the top disc from one peg to another
Transition model	The selected disc moves to a legal destination peg
Goal test	All discs are stacked on peg C in correct order
Path cost	Each legal move costs 1

Rules: Move only one disc at a time. Never place a larger disc on a smaller disc.

ii. State space for the first three levels

Let the state (x, y, z)represent the positions of the smallest, middle, and largest discs, respectively.

Level 0:

(A,A,A)

/ \

Level 1: (B,A,A) (C,A,A)

| |

Level 2: (C,A,A) (B,A,A)

(C,C,A) (B,B,A)

(B,A,A) (C,A,A)

The diagram shows representative legal moves, including reversals. In a complete search tree, each legal action is generated, and repeated states can be avoided using visited-state tracking.

iii. Minimum sequence of moves

For three discs, the minimum number of moves is:

$$2^n - 1 = 2^3 - 1 = \boxed{7}$$

The optimal sequence is:

Move	Disc	From	To
1	Small	A	C
2	Medium	A	B
3	Small	C	B
4	Large	A	C
5	Small	B	A
6	Medium	B	C
7	Small	A	C

All discs are now on peg C in the correct order.

General formula for $n$discs:

$$T(n) = 2T(n-1) + 1$$

with $T(1) = 1$. Solving this recurrence gives:

$$\boxed{T(n) = 2^n - 1}$$

The time complexity is $O(2^n)$.

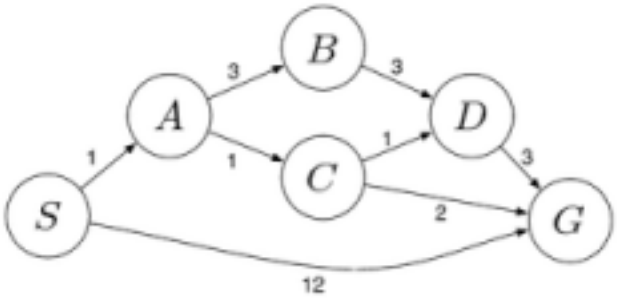
iv. DFS or BFS — which is more suitable?

BFS (Breadth-First Search) is suitable when the objective is to guarantee the minimum number of moves, because every move has equal cost and BFS explores states level by level.

- BFS guarantees the shortest solution but requires substantial memory.
- DFS uses less memory but does not generally guarantee the shortest solution.
- For larger instances, the recursive Tower of Hanoi solution is more efficient than explicitly exploring the entire search tree.

15.A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



i .Graph edges and costs

Route	Cost
S → A	1
S → G	12
A → B	3
A → C	1
B → D	3
C → D	1
C → G	2
D → G	3

ii. Apply Uniform Cost Search

UCS expands the node with the lowest cumulative path cost $g(n)$.

Step 1: Start at S
Frontier: A(1), G(12)

Step 2: Expand A(1)
Frontier: C(2), B(4), G(12)

Step 3: Expand C(2)
Frontier: D(3), B(4), G(4)

Step 4: Expand D(3)
Frontier: B(4), G(4), G(6)

Step 5: Expand B(4)
Frontier: G(4), D(7)

Step 6: Select G(4)
Goal reached!

At C, the direct route to G has cost $2 + 2 = 4$. The route through D costs $2 + 1 + 3 = 6$, so the direct route is cheaper.

iii. Find the least-cost path

The least-cost path is:

$$S \to A \to C \to G$$

Its total cost is:

$$\begin{aligned} \text{Cost} &= 1 + 1 + 2 \\ &= \boxed{4} \end{aligned}$$

Other possible routes include:

- $S \to A \to B \to D \to G$: cost $1 + 3 + 3 + 3 = 10$.
- $S \to A \to C \to D \to G$: cost $1 + 1 + 1 + 3 = 6$.
- $S \to G$: cost 12.

Final result: UCS finds the optimal route $S \to A \to C \to G$with a minimum total cost of 4.

iv. Why is UCS suitable?

- It expands paths in increasing order of cumulative cost.
- It handles different positive edge weights.
- It guarantees an optimal solution when step costs are non-negative.
- It is more appropriate than ordinary BFS when route costs differ.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem